APPENDIX K

Star-camera algorithms

K.1. Space story

I was working at GE Astro Space when I came up with the idea of the Polaris sensor, a bright-star sensor that would look North on a geosynchronous communications satellite. GE Pittsield had developed charge-injection device (CID) imagers that did not suffer from many of the problems inherent in charge-coupled device (CCD) imagers. I got IR&D money for the sensor and we began design work. A major problem turned out to be reflections from the solar wings that might blind the sensor due to reflections from the lens barrel. We came up with the idea of tilting it offaxis. Unfortunately, it was not a Polaris sensor. Eventually the program was canceled.

K.2. Introduction

Star cameras are a popular sensor for spacecraft-attitude determination. The idea is to measure the positions of stars in the imager frame and compare those with a star catalog. If the sensor measures at least three stars then 3-axis attitude determination can be done with one sensor. Often the sensor is coupled with a gyro. The star camera provides an initial attitude measurement and then corrects for gyro drift.

This chapter will discuss the process of going from an image of stars to locations of those stars in the field-of-view. More details on how they are then used for attitude determination is found in the estimation chapters.

K.3. Center-of-mass star centroiding

The first step is to get a coarse location of each star in the focal plane. The camera deliberately defocuses the stars so that they are spread over multiple pixels. This allows the algorithm to find a centroid with an accuracy that is not limited by the pixel size.

This section describes the initial "coarse" centroiding performed on an image. This centroiding is sufficiently accurate for star identification. This is a standard technique for centroiding stars in a star tracker. Fig. K.1 shows the steps required in the algorithm. First, the sky background must be modeled. This can be done with a Gaussian fit to the image histogram. The resulting background level b and standard deviation R allow for the pixelmap to be thresholded, and all pixels below the noise floor to be zeroed out. The remaining pixels comprise the imaged stars. In practice, the algorithm may be able to use a fixed threshold once the inspace background is modeled during on orbit

checkout. The threshold is based on the range of stellar magnitudes in the catalog that is used for star identification.

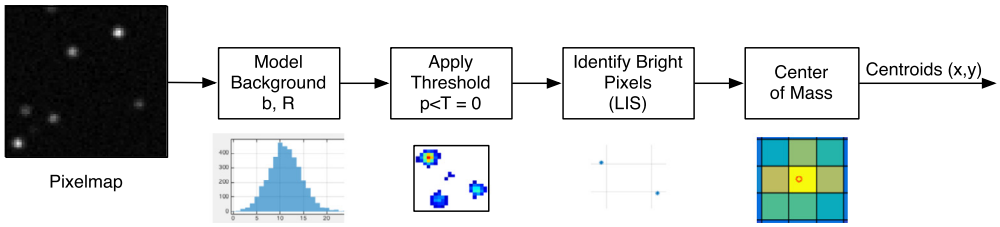

Figure K.1 Coarse centroiding steps.

A blobification routine can group the remaining pixels into blobs and find the brightest pixel in each. The reason it is called "blobification" is because the star images are not regular shapes. This is required in "lost-in-space" operation when the spacecraft has no estimate of its attitude. The routine eliminates blobs that are smaller than a specified minimum number of pixels. During tracking mode, when stars have already been identified and the spacecraft is rotating slowly, not all of the above steps may be required. Windows of the frame around the previously identified stars may be analyzed instead of the entire pixelmap and a local threshold may be applied.

K.3.1 Background noise

The background noise generates a Gaussian-like distribution of low pixel levels throughout the frame. This must be removed from the signal. One method is to fit a Gaussian to the histogram of the pixel counts, using the principal maximum of the histogram as the initial guess for the height of the curve [1]. Newton's method of nonlinear least squares usually gives a solution in two iterations. From the histogram, n_k are the discrete counts for each discretized intensity value I_k, see Fig. K.2. The Gaussian is expressed as,

$$f(I) = H \exp[-(I-b)^2/2\sigma^2] \tag{K.1}$$

The initial value for the mean background level b is the I_k with the maximum number of counts, i.e., the mode of the histogram. H is the height of the curve. When performing the least-squares fit it is best to scale the histogram counts n_k by the maximum value N. In this case the initial value for H is 1. σ can be initialized to the square root of b.

The least-squares δ for each iteration is

$$\delta = -(J^T J)^{-1} J^T f(I_k) \tag{K.2}$$

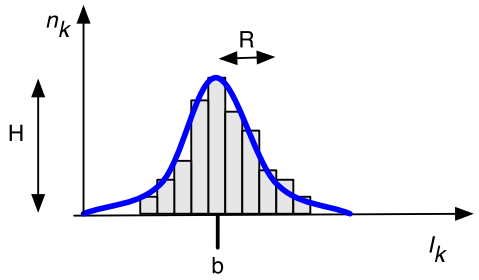

Figure K.2 Pixel histogram. *b* is the mean background level.

where the Jacobian matrix J of partial derivatives is

$$J_{ij} = \frac{\partial f_i(I)}{\partial x_j} \tag{K.3}$$

for the state elements $x = [b\, H\, \sigma]$ and an array of I_k around the initial value. In this case we can calculate the partial derivatives of the Gaussian distribution directly and avoid a numeric Jacobian,

$$\frac{\partial f}{\partial H} = \exp[-(I-b)^2/2\sigma^2] \tag{K.4}$$

$$\frac{\partial f}{\partial b} = -\frac{H}{\sigma^2}(I-b)\exp[-(I-b)^2/2\sigma^2] \tag{K.5}$$

$$\frac{\partial f}{\partial \sigma} = -\frac{H}{\sigma^3}(I-b)^2\exp[-(I-b)^2/2\sigma^2] \tag{K.6}$$

Two or three iterations are sufficient to calculate b and σ to several digits.

Once the sky-background level b is known, the pixel array can be thresholded as follows:

$$I(x,y) = I(x,y) - b \quad \text{if} \quad I(x,y) \geq T \tag{K.7}$$

$$I(x,y) = 0 \quad \text{if} \quad I(x,y) < T \tag{K.8}$$

The threshold level T must be selected on the basis of b and σ, for example at the 3σ level, this will be $T = b + 3\sigma$.

Fig. K.3 shows the effects of thresholding a simulated image. The image is first generated by integrating the Gaussian point-spread function for a set of sources, as shown on the left. Noise is then added using a Poisson process, center. A fit to the background of the image is performed and a threshold is applied, on the right. You can see that the edges of the stars becoming ragged, losing some signal to the background noise, and the dimmest star (upper left) is broken up into small pieces.

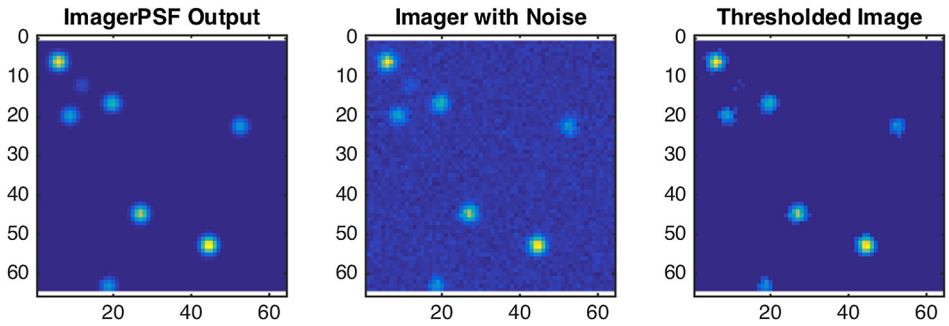

Figure K.3 Effect of thresholding a simulated image. Thresholding, on the right, removes the noise.

K.3.2 Creating a star blob

The blobification routine performs a search on the pixels left after thresholding. If pixels are adjacent to any other remaining pixels, they are added to the blob. A blob consists of the following parameters:

- Bounding box: top/bottom row indices, left/right column indices;
- List of pixels in the blob;
- Location and value of brightest pixel in the blob.

For each pixel in the thresholded image, if it is adjacent to one of the existing blobs, it is added to that blob's list. If it is brighter than the brightest pixel, the brightest pixel is updated. If the pixel is not in any existing blobs, a new blob is created.

Example blobs are shown in Fig. K.4. The white box is the bounding box.

K.3.3 Center-of-mass

The center-of-mass algorithm is a simple weighted average of the pixels in a region of interest centered on a bright pixel. In the most general form of the algorithm, a local noise floor can be computed from the ring of pixels surrounding the region of interest. This step can be skipped if the image is already thresholded as in Section K.3.1.

The center-of-mass algorithm has the following steps:

1. Input is a square region of interest (ROI) centered around the brightest pixel.
2. Calculate a noise threshold from the pixels surrounding the ROI (optional).
3. Compute the total intensity of the ROI (above the noise floor).
4. Compute the centroid by weighting each pixel by its intensity.

In a tracking implementation, the first step is not needed; the center pixels would be an input using the previous frame along with estimation of the satellite's rotation. The first step is therefore part of the lost-in-space (LIS) solution. The region of interest and the edge used for calculating the noise floor are shown in Fig. K.5.

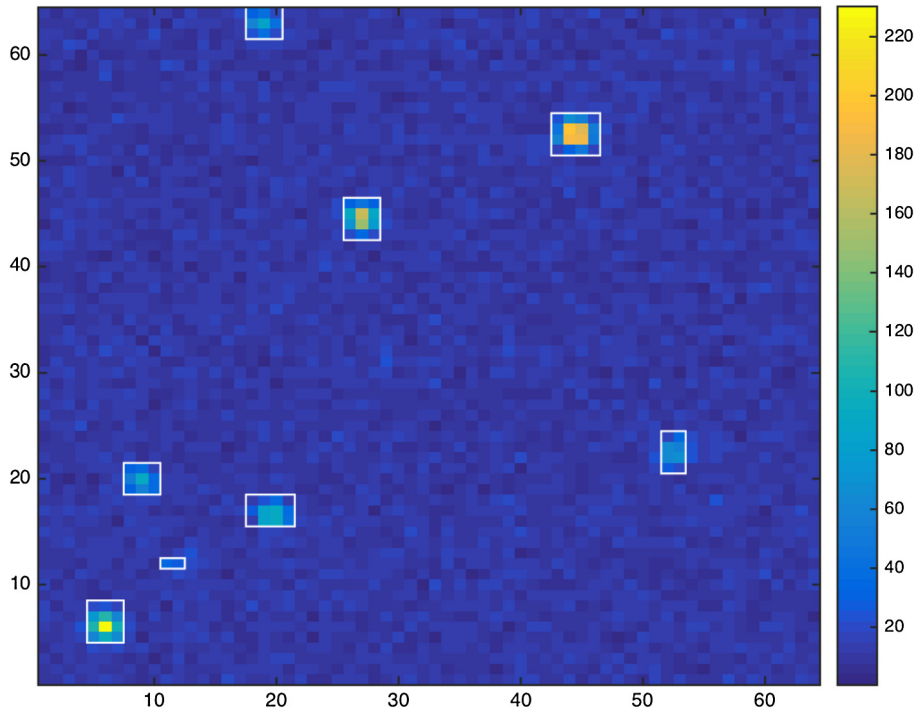

Figure K.4 Blobify routine results. Seven blobs, which are potential stars, are identified.

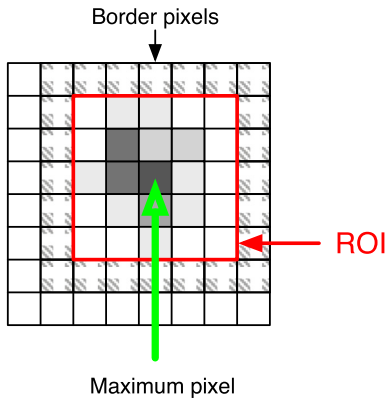

Figure K.5 Region of interest with border.

The noise floor η is an average of the N pixels around the edge of the region of interest (ROI). j is the column value of the pixel and i is the row value. This may be substituted with a constant noise value or skipped if the noise has been subtracted in a

prior step,

$$\eta = \sum_{edge} p(j,i)/N$$

The total intensity is the sum of the pixels in the ROI after subtracting the noise threshold.

$$I = \sum_i \sum_j p(j,i) - \eta$$

The x and y values of the centroid are then calculated using a center-of-mass calculation of the pixels.

$$x = \frac{\sum\sum j(p(j,i)-\eta)}{I} + 0.5$$
$$y = \frac{\sum\sum i(p(j,i)-\eta)}{I} + 0.5$$

where x and y are defined with the origin at the corner of the image. The origin needs to be transformed to the center of the frame before the centroids can be used by the star-identification algorithm.

K.4. Star identification

A pyramid star ID algorithm is used for star identification. Identifying a pyramid of stars virtually assures a correct lost-in-space identification, and allows the algorithm to exit the search once a pyramid is found. The algorithm first uses triad checking to find a candidate triad, then checks the remaining measurements for a self-consistent pyramid match, i.e., a unique match with all three stars in the triad. This dramatically reduces the processing time on a new image, from 0.2 second to identify stars from 15 centroids to only 0.02 seconds (in MATLAB®). This correlates well with the reference for a fast pyramid technique [2]. It has been tested on orbit on the High Energy Transient Explorer (HETE) spacecraft.

K.4.1 Catalog processing

The star-catalog processing is shown in Fig. K.6. We start with the Hipparcos catalog. Stars are selected that are above the minimum desired magnitude, which is based on the camera field-of-view and planned exposure time. Then, any stars that will be so close together as to be within the same region of interest used for centroiding, must be removed. The field-of-view is then used to generate the near matrix of star pairs, or all pairs of stars that are close enough to appear together in an image. Finally, the resulting star pairs are sorted and the k-vector parameters stored for fast lookup.

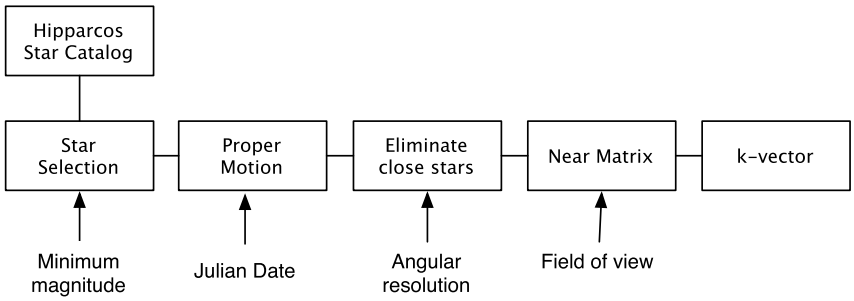

Figure K.6 Star-catalog processing. This reduces the catalog to a manageable size and makes star identification easier.

The catalog can be further optimized by restricting the number of stars kept in the densest regions of the sky [3]. If there are more than a set number of stars in a field-of-view (FOV) region, for example 12 stars, then only the brightest 12 are kept. This reduces the number of star pairs in the near matrix and makes star identification faster. The Hipparcos catalog has 13 943 stars with a minimum visual magnitude of 7, which would have a minimum of 19 stars in a 15 degree field-of-view. A magnitude of 6.5 results in 7982 stars and a minimum of 9 in the field-of-view. If we optimize a catalog using magnitude 7 stars, we can have a catalog with just 4436 stars and a minimum of 11 in the field-of-view (using an increment of 10% of the field-of-view to scan the sky). This is shown in Fig. K.7; note that the shape of the milky way is not readily apparent and that the star density is fairly uniform.

K.4.2 Sorting star pairs with *k*-vector

The *k*-vector range search technique allows a range of values from the near matrix to be extracted without searching, by using indexing. The star angles of the star pairs in the near matrix are sorted in ascending order, producing an array of star angles s, and an additional integer index value is assigned to each pair. A slope and intercept is calculated for an equivalent straight line through the data. The indices of s can then be calculated for a given star-angle range and all values within the range easily retrieved. Practically, the index k gives the number of elements in the angle array below the value dictated by the equivalent line.

The equation of the straight line connecting the minimum and maximum angles, extended by a small delta ξ is

$$z(x) = mx + b$$

where

$$m = \frac{s_{max} - s_{min} + 2\xi}{n - 1} \tag{K.9}$$

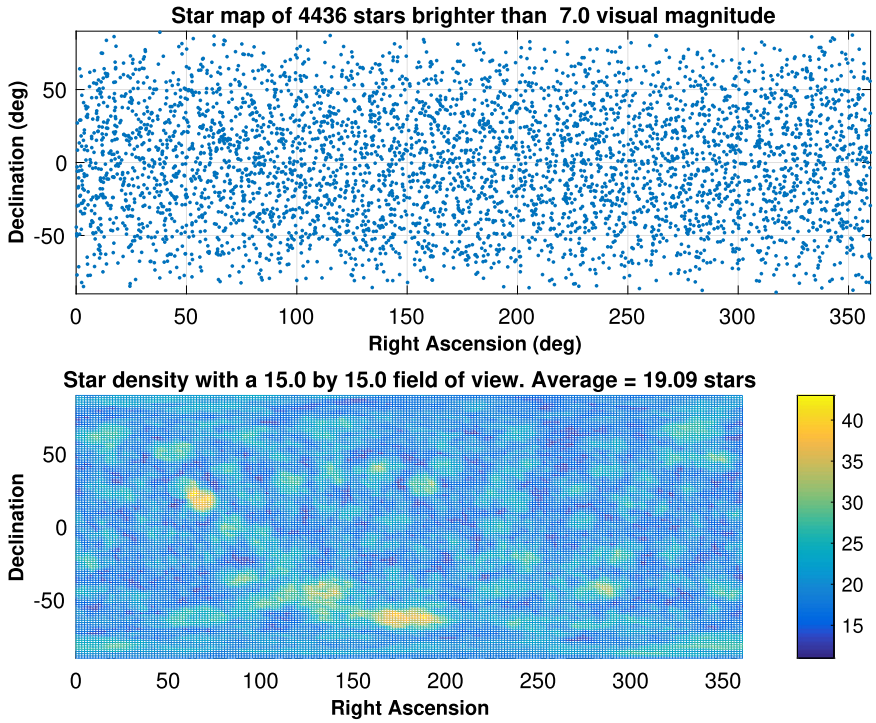

Figure K.7 Reduced star catalog. The field-of-view is used to reduce the catalog size. Only the brightest stars in a given region are kept.

$$q = \gamma_{min} - m - \xi \tag{K.10}$$

$$\xi = \epsilon \max[s_{min}, s_{max}] \tag{K.11}$$

and ϵ is the relative machine precision, and s_{min} and s_{max} are the minimum and maximum values of the input data s. That is, z is the line connecting the points $[1, s_{min} - \zeta]$ and $[n, s_{max} + \zeta]$ where there are n star-pair angles in s.

The integer vector k is constructed as

$$k(i) = j \quad \text{if the } j \text{ index satisfies} \quad s(j) \leq z(i) < s(j+1)$$

starting with $k(1) = 0$.

The evaluation of two indices k_{start} and k_{end} identifying the data within a given range $[\gamma_a, \gamma_b]$ involves calculating the floor and ceiling values, j_a and j_b,

$$j_a = \left\lfloor \frac{\gamma_a - q}{m} \right\rfloor \quad \text{and} \quad j_b = \left\lceil \frac{\gamma_b - q}{m} \right\rceil$$

With these indices it is possible to look up the indices of s,

$$k_{start} = k(j_a) + 1 \quad \text{and} \quad k_{end} = k(j_b)$$

which guarantees that

$$s(k_{start}) < \gamma_a \quad \text{and} \quad \gamma_b < s(k_{end})$$

Therefore the possible matching star pairs are those in the matrix of pairs between k_{start} and k_{end}. Note that it is possible for the range to include extra points beyond the inputs, which are still within the range of the closest line indices; these can be eliminated with a linear search.

The script KVectorDemo produces a demo with the plots in Fig. K.8. This is for a simple set of random numbers on the interval (0,1). The plot on the left shows the steps in k, in a stair plot, as compared against a linear progression. For example, at index 4, k jumps from two to four, indicating that there are four data points with values below the line value at that index. On the right, the data is printed against the straight-line fit to the endpoints, with a range calculated of 0.5 to 0.75. The range is marked with the magenta solid lines. The data points that are circled are the values returned as in the desired range. We can see in this particular example that one data point below 0.5 was returned, because it is also above the value of the closest line point, 0.4642. The line points bracketing the range are indicated by the black dashed lines.

When using k-vector with measured star pairs, the angle range will be the measured angle $\theta \pm \delta$. This should be a high confidence interval such as 4σ. This could be a constant value for the entire pixel array or calculated based on the star locations or angle.

K.5. Fine centroiding

The concept for fine centroiding involved a numerical fit of an integrated 2D point-spread function (PSF) against the pixel values for each star. However, the defocused Airy PSF for a single wavelength does not resemble the PSF for the full-wavelength spectrum of a star, which in fact resembles a Gaussian. A Gaussian can be readily fit to the pixel image in separate X and Y directions, in a procedure known as digital centering of the marginal distributions [4]. This algorithm is used in astrometry. This performs very well against the numerical fit of a 2D Gaussian to a simulated image smeared with Poisson noise. This effectively corrects errors from the coarse centroiding that are larger as stars are located more to the outside corners of the pixel.

The digital centering means fitting a Gaussian to the pixel sums in each coordinate axis of the focal plane. This can be accomplished using a Newton–Gauss method (least squares) with analytic derivatives in just 2 or 3 iterations. The fit gives a measure of both the star-centroid location and the PSF width (Gaussian standard deviation σ). This

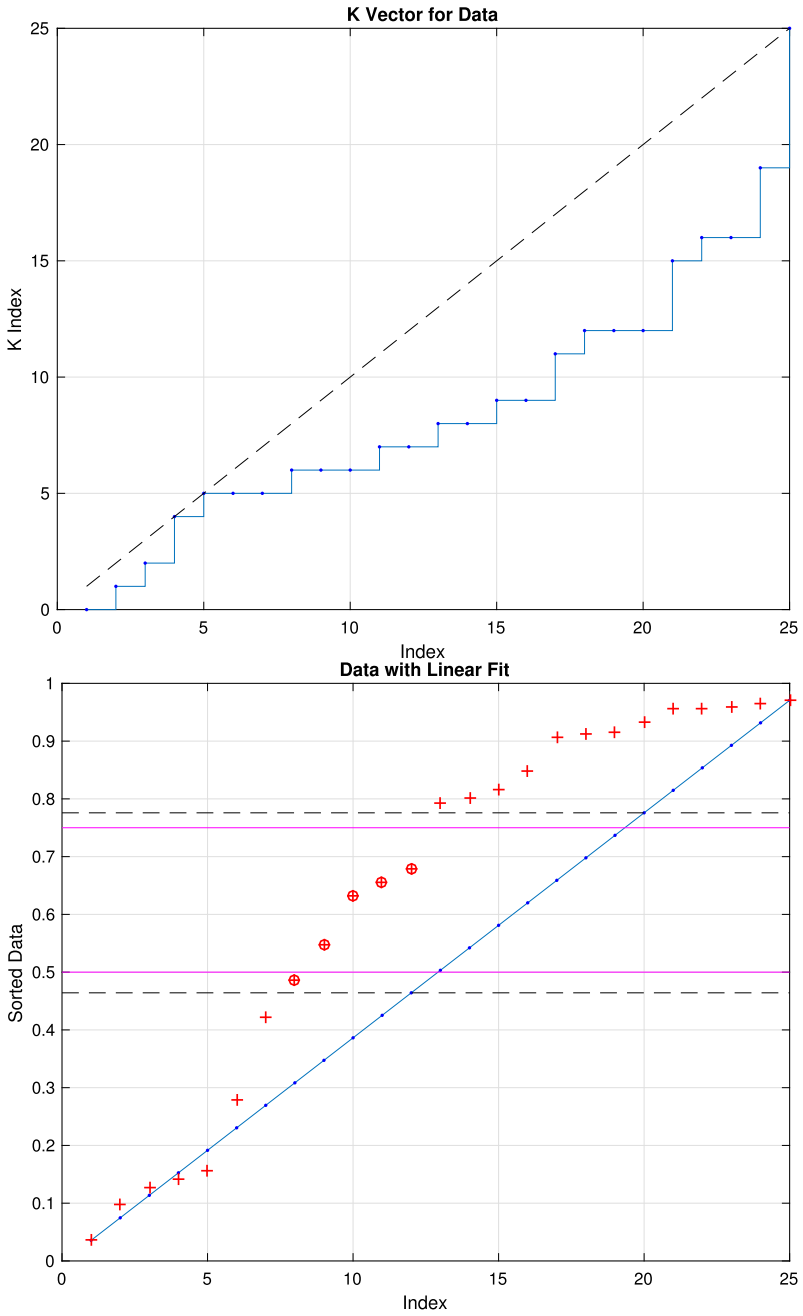

Figure K.8 K-vector example in MATLAB. The plot on the left shows the steps in k. The right plot shows the data display against a straight-line fit.

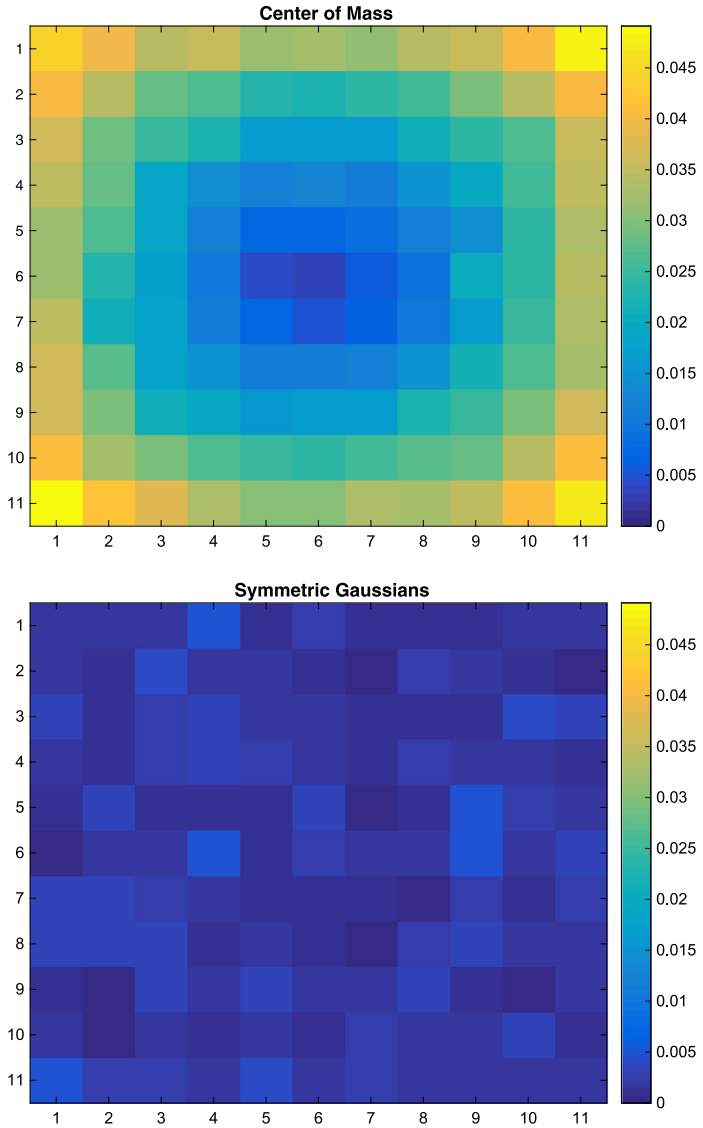

Figure K.9 Center-of-mass centroiding compared against Gaussian fitting to the marginal distribution when the centroid moves throughout the pixel. The scales of the color bars in both images are the same.

is essentially the same method used to fit the sky background to the image histogram in Section K.3.1. In implementation, it is as fast as the center-of-mass method, while eliminating errors when the centroid is towards the corner of the frame. The following two plots in Fig. K.9 show a comparison of the fractional centroid error as the centroid

is moved throughout a pixel, with the Gaussian centering showing a clear improvement. This is for a PSF simulated using a Gaussian and smeared with Poisson noise.

Fig. K.10 shows an example of the digital centering method. The sums of the values in each direction are shown on the bar chart. The Gaussian fit is shown on top in red. Note that the height of the fit is greater than the height of the data. The source pixelmap is shown on the right.

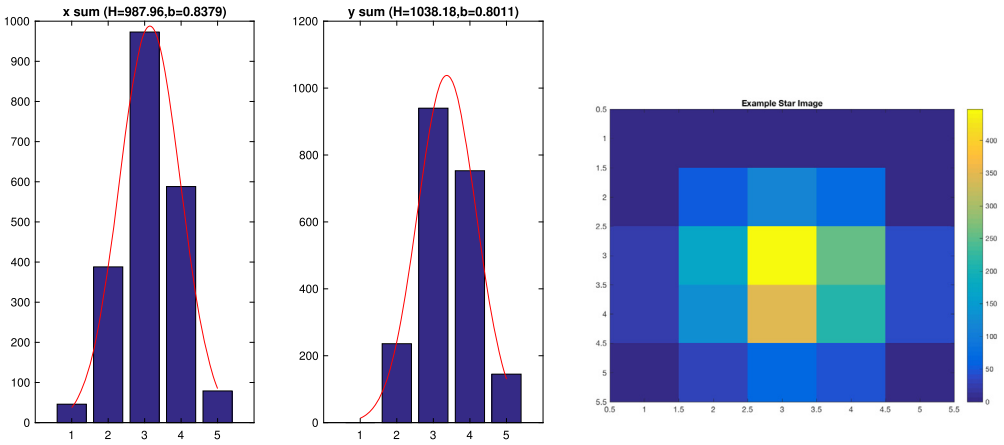

Figure K.10 Digital centering example.

References

[1] A. Bijaoui, Sky background estimation and application, Astronomy & Astrophysics 84 (1–2) (April 1980) 81–84.
[2] D. Mortari, M. Samaan, C. Bruccoleri, J. Junkins, The pyramid star identification technique, Navigation 51 (3) (September 2004) 171–183.
[3] J.D. Vedder, Star trackers, star catalogs, and attitude determination: probabilistic aspects of system design, Journal of Guidance, Control, and Dynamics 16 (3) (May–June 1993).
[4] R.C. Stone, A comparison of digital centering algorithms, The Astronomical Journal 97 (4) (April 1989).